Adaptive Schedules and Exact Incremental Packing

Improvements beyond the current SubexponentialSAT certificates

Research prepared for Vladimir Reshetnikov
with ChatGPT

October 26, 2026 (Pacific time)

Abstract

We review seven refinement packets in ProveIt's SubexponentialSAT project and all eight research archives in its incoming directory at commit `d098ba436b33`. The meaningful baseline includes compatible combinations of these proposals. We give a rational multistage schedule which, combined with their asymmetric packing, sharp Hermite precision, and full crossing-block Cauchy modes, has normalized polynomial-space contraction coefficient $C_{\mathrm{poly}} = 63.781173756244\ldots$, below the comparable prior fixed-alphabet boundary certificate $66.192723693577\ldots$. A separately costed three-stage schedule for the existing cached mode-graph engine gives time coefficient $C_{\mathrm{cache}} = 37.59170255554\ldots$ and space coefficient $S_{\mathrm{cache}} = 33.6230409217\ldots$. We also implement two exact packing improvements. Incremental matching potentials reduce the labeling update work from $\Theta(VMQ^3)$ to $O(MQ^2 + VQ)$ rational operations, with explicitly charged initialization and additional state. A common-denominator column implementation improves the already cached rational reference in reproducible benchmarks. Both preserve the original deterministic labels and exact potential values. The coefficients multiply $s/\sqrt{\log_2 s}$ for a declared tensor interface, not the SAT variable count. The general SAT logarithmic order is unchanged. The analytic results retain the supplied framework's exact algebraic identities; the executable artifacts are packing kernels, not a complete SAT solver.

Deliverable. The archive contains this article's full LaTeX, the compiled PDF, exact certificate checkers, portable implementations, verbatim benchmark references, source identities, and raw measurement records.

Contents

1 Results and the comparison that they support 3

2 Repository review and the complete comparison set 4

2.1 The seven placed packets . . . 5

2.2 The eight incoming packets . . . 5

3 The exact interfaces being combined 6

3.1 Asymmetric packing and the fresh side . . . 7

3.2 Full crossing-block modes preserve the interface . . . 7

3.3 Sharp Hermite precision with a thin gap . . . 8

3.4 The resulting local cost . . . 8

4 A rational multistage schedule with polynomial space 9

4.1 Parameters and exact margins . . . 9

4.2 Summing the recursion and certifying the coefficient . . . 10

4.3 Comparison with the stronger prior family . . . 11

5 A faster schedule with subexponential memory 11

5.1 The fully charged local recurrence . . . 11

5.2 An exact three-stage schedule . . . 13

5.3 Time and memory, including the entire tail . . . 13

6 Two exact improvements to executable packing kernels 14

6.1 Incremental products for matching potentials . . . 15

6.2 Common-denominator integer scores for column balancing . . . 16

7 Reproducible tests and measured performance 17

7.1 Exact validation . . . 17

7.2 Benchmark protocol . . . 18

8 Integration, remaining limits, and further research 19

8.1 How the results fit into the existing framework . . . 19

8.2 Preserve the best applicable previous branch . . . 19

8.3 Why the general logarithmic order has not changed . . . 19

8.4 Specific next research questions . . . 20

1 Results and the comparison that they support

The request was to review the recent proposals and improve on their combined frontier. Comparing only to the older 648.44 certificate would miss most of the subsequent progress. The new incoming archives supply asymmetric degree bounds, sharper crossing modes, nearly lossless packing, thin precision gaps, and an optional algorithm that spends more memory to reduce time. We account for these together before stating an improvement.

Let $s \geq 2$ be a normalized root budget, and let $\mathcal{L}$ be the total explicit graph and tensor-description length. The interface throughout the numerical comparisons is a bipartite Boolean tensor network with at most s wires and maximum degree at most $\left\lceil s^{9/10} \right\rceil$. Its entries belong to

$$(\mathbb{Z}/2^{P_0}\mathbb{Z})[V]/(V^{\ell}), \qquad 1 \leq P_0, \ell \leq s.$$

Original entries must be evaluable in $(s+\mathcal{L})^b \exp(As^{9/10})$ time and $(s+\mathcal{L})^b$ space, for fixed constants A, b. Any fixed $O(s)$ allowance for P_0, ℓ gives the same leading results. All statements retain the source's creation-time precision convention and exact contraction identities.

Theorem 1.1 (Two explicit improved certificates)**.** *Under this interface and the imported exact identities specified in Section 3, the following deterministic contraction bounds hold.*

1. *In polynomial space,*

$$T(s,\mathcal{L}) \leq (s+\mathcal{L})^{O(1)} 2^{(C_{\mathrm{poly}}+o(1))s/\sqrt{\log_2 s}}, \qquad C_{\mathrm{poly}} = 63.781173756244\ldots < 64.$$

2. *With subexponential space,*

$$T(s,\mathcal{L}) \leq (s+\mathcal{L})^{O(1)} 2^{(C_{\mathrm{cache}}+o(1))s/\sqrt{\log_2 s}},$$
$$S(s,\mathcal{L}) \leq (s+\mathcal{L})^{O(1)} 2^{(S_{\mathrm{cache}}+o(1))s/\sqrt{\log_2 s}},$$

where

$$C_{\mathrm{cache}} = \frac{\pi}{\ln 2}\sqrt{\frac{300000}{4361}} = 37.59170255554\ldots, \qquad S_{\mathrm{cache}} = \frac{\pi}{\ln 2}\sqrt{\frac{240000}{4361}} = 33.6230409217\ldots.$$

In particular, eventual coefficients 38 *for time and* 34 *for space are valid.*

The first certificate is approximately 3.64% smaller in its leading coefficient than 66.192724…, the prior asymmetric fixed-alphabet boundary combined with the full-block modes. It is also below the strictly feasible displayed composition, about 66.6051. This is a comparison of upper-bound certificates. It does not prove that every execution of an older algorithm takes longer, nor a ratio of actual SAT runtimes.

The second statement must have its own comparison. Composing the reviewed cached engine with near-unit asymmetric packing already gives a coefficient approaching 38.167745697 with alphabet eight. A further two-stage continuous comparison gives approximately 37.8906. Our explicit rational three-stage schedule is below both. These comparisons are specified schedule families; we do not claim a global optimum over every possible reoptimization.

Certificate in the same high-degree interface	Time	Space
Asymmetric packet, displayed	76.9090	polynomial
Asymmetric displayed schedule + full-block modes	66.6051	polynomial
Prior fixed-alphabet boundary + full-block modes	66.1927	polynomial
New rational multistage schedule	**63.7812**	polynomial
Composed cached engine, alphabet-eight boundary	38.1677	33.9927
New rational three-stage cached schedule	**37.5917**	**33.6230**

An exponential space entry is a coefficient on the same $s/\sqrt{\log_2 s}$ scale. Boundary rows are infima approached with inward slack, not asserted finite-size parameter settings.

The practical result is independent of those enormous asymptotic thresholds. For the newest matching-potential prototype we preserve every assignment, matching, threshold, load, and selected exact potential. The optimized algorithm avoids repeated products. A second exact-integer routine preserves the column-labeling reference, including its tie-breaking. Section 7 gives measured timings and the exact scope of the tests.

2 Repository review and the complete comparison set

The review is frozen at ProveIt commit `d098ba436b33df19365a44dd178694ff52845d19`. Its relevant corpus consists of seven refinement packets already placed in *SubexponentialSAT*, eight further archives in the incoming directory, and two primary manuscripts. The supplied source manifest records the paths, Git blob identities, archive hashes, and retrieved source members. This fixed snapshot makes the comparison reproducible even as subsequent reports arrive.

Closing relocation check. The later head `474ddba696a5d385c091c23fb2995f69edd13e00` places these same eight incoming packets in the topic project's Research subdirectories; its incoming directory contains only the index. We compared all 211 retained payload files with the already reviewed archive members: every Git blob identity matches. The original seven packet subtrees are also unchanged. Thus the closing commit introduces no new proposal or changed research payload. The delivery records the new paths while retaining the intake pin for the archive citations.

What the recent commits changed. Commit `2b925d8800cd` placed the original seven packets in the topic project. Commit `3e096089911f` vendored the OpenAI manuscript sources; `45233238aebf` summarized the then-current claims without rechecking them. Arrival `3e57705cf14e` added seven new archives, which therefore lie outside that summary. The final reviewed arrival, `d098ba436b33`, added the matching-potential packet. The present review includes both arrival batches. The source preprint is pinned at OpenAI `7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`; earlier packets using `42907c9e999c` inspect the same manuscript bytes[1, 2].

Audit status. We reviewed the statements, source identities, and proof interfaces relevant to the comparison, checked the compatibility used in this article, and independently checked the new schedules and executable kernels. We did not rebuild the upstream Lean development or independently reprove every part of either primary manuscript. The following inventory therefore distinguishes ordinary mathematical refinements within an inherited framework from missing algorithmic hypotheses. Finite Python checks support implemented identities; they do not establish an asymptotic theorem by sampling large parameters.

2.1 The seven placed packets

These packets establish the earlier analytical framework. Their constants and degree refinements must be accounted for before claiming a further gain. The column headed “limit” identifies the scope needed when composing them.

Packet	Relevant contribution	Limit
Factorial Cauchy[4]	Integral separated modes with weights $3j + 1 - 2\,\mathrm{popcount}(j)$; leading entropy factor $1/\sqrt{3}$.	Same logarithmic order.
Contraction refinements[5]	Exact Hermite taper, precision $6MD/u$, coefficient 648.4411, and streamed coefficient extraction.	Normalized tensor certificate; no SAT benchmark.
Precision refinement[6]	Sharp recovery losses, taper optimization, and a structured mode representation.	The $n \log\log n/\log n$ target assumes an unproved summation algorithm.
Density improvement[7]	Uniform dependence on $\min\{\log(2 + m/n), \log\log n\}$.	Improves selected density regimes.
Density refinement[8]	Related density bound, precision refinement, and an odd-Cauchy Smith profile.	Does not alone construct the later full-block recursive basis.
Scalar parity lift[9]	Scalar encoding and incidence-degree cap $k(n + a)$ instead of $k(2n + a)$.	Its faster second contraction needs an unsupplied compact factorization.
Asymptotic analysis[10]	Representation-specific barriers and explicit missing interfaces for stronger orders.	Establishes no stronger unrestricted bound.

The common density-sensitive envelope is

$$(n+m)^{O_k(1)} 2^{O_k\left(\frac{n}{\sqrt{\log n}} \min\{\log(2+m/n), \log\log n\}\right)}.$$

For example, when $m = \Theta(n \log\log n)$ its numerator is $n \log\log\log n$. This is an established earlier refinement within the source framework, not a new consequence first obtained here. The unrestricted and fixed-density logarithmic orders are unchanged by the present schedules.

2.2 The eight incoming packets

The next table lists each archive's own displayed general contraction coefficient, where supplied. All numerical entries use the declared $s/\sqrt{\log_2 s}$ normalization. They are not mutually optimized combinations, and the smallest displayed number is consequently not the full baseline.

Packet	Relevant contribution	Displayed certificate
Balanced packing[11]	Two-stage whole-vertex labeling, near-unit padding, thin precision gaps, integer capacity schedule.	94.7482; polynomial space.
Higher moments[12]	Higher-moment derandomization, orbit padding, and degree-sensitive specializations.	94.7482; polynomial space.
Packet refinement[13]	Exponential conditional potential and a cached-suffix implementation of column labeling.	95.6639; polynomial space.
Recursive refinement[14]	Reciprocal-domain modes of asymptotic slope 64/21, with sharper padding and gaps.	133.2074; polynomial space.

Packet	Relevant contribution	Displayed certificate
Joint Cauchy[15]	Full-orientation slope-four modes; exact-weight indexing; cached group tables and Strassen contraction.	319.6858 in polynomial space; 225.5971 time with space coefficient 196.1994.
Asymmetric packing[16]	Fresh-side degree invariant, asymmetric load bounds, and exact crossing fraction.	76.9090; polynomial space.
Round-two refinement[17]	Matching potential, thin gaps, and alphabet 37 at the root followed by 8.	84.0663; polynomial space.
Latest matching packet[18]	Exact-rational matching potential, integral schedule, and the reference accelerated by our first kernel.	94.7482; polynomial space.

Specialized results remain separate. The higher-moment report already derives degree-sensitive coefficients, including approximately 31.2365 for bounded-degree input networks. The asymmetric packet gives 31.0641 when the initial maximum degree is at most $\left\lceil s^{1/12} \right\rceil$. Those assumptions are stronger than the general $s^{9/10}$ interface used for our headline comparison. Generated packet vertices still acquire degree up to $2L$; bounded input degree does not remain bounded automatically. Neither lowering the initial degree nor varying the alphabet by depth is claimed as an invention of this article.

Compatible combinations determine the baseline. The joint Cauchy basis preserves source and target groups and the mode-family multiplicity. It therefore combines with asymmetric packing and its smaller fresh-side degree. Replacing slope three by slope four multiplies that packet's entropy coefficient by $\sqrt{3/4}$, giving the stronger comparison in Section 1. The cached mode-graph engine also combines with near-unit padding, but consumes exponential space. We cost that combination separately. Our new work is an explicit feasible schedule and implementation improvement beyond these identified combinations, with no claim of global optimality over every possible schedule or algorithm.

The separate manuscript and conditional frontier. The 49-page manuscript attributed to Claude uses a different crossing construction and states the same general logarithmic order with an unspecified leading constant[3]. Its formalization claim is not a public proof build checked here. Consequently its runtime cannot be numerically ordered against the normalized constants in our tables. Likewise, an unproved $n/\log n$-type target is not an implemented competitor that a constant improvement can surpass. These limits coexist with concrete progress: the present certificates improve the stated compatible bounds, and the supplied kernel measurements compare executable implementations with exactly matching outputs.

3 The exact interfaces being combined

The new schedules change how work is allocated across recursive levels. Their correctness uses the same deformation, recovery, and child-factorization identities as the supplied preprint and refinement packets. We spell out the interfaces to make the combination checkable. We do not substitute the weaker formalized endpoint for the stronger manuscript bound, nor infer a formal proof from finite tests.

3.1 Asymmetric packing and the fresh side

At an active call with M wires, choose a packet length $L = 2^a$, an alphabet $r \geq 3$, and $Q = r^a$ whole-vertex labels on each side. Let R, C bound the row and column degrees. The asymmetric packing theorem gives labels with maximum ordered-pair load at most

$$\left(1 + \frac{2}{h_0 - 1}\right)\left(\frac{M}{Q^2} + \frac{R}{Q}\right) + 2(h_0 C + 1)\ln Q, \qquad h_0 \geq 2. \tag{1}$$

This is an imported theorem of the asymmetric packet [16]. Its construction is useful here: greedily balance row stub counts, then assign column labels by exact conditional expectations of $\sum_{x,y} z^{X_{xy}}$, with $z = 1 + 1/(h_0 C)$. The average of the candidate expectations equals the previous expectation, so a minimizing label never increases it. Convexity of z^w on $0 \leq w \leq C$ proves (1). No real logarithm is evaluated to make an algorithmic choice.

Write $\lambda = \lceil \log_2 s \rceil$ and take $h_0 = \max\{2, \lambda\}$. Rounding the maximum load upward to a multiple of L yields a common padded mass $P = (1 + o(1))M$ whenever, uniformly over active calls,

$$\frac{QR}{M} = o(1), \qquad \frac{Q^2 C \lambda^2}{M} = o(1), \qquad \frac{LQ^2}{M} = o(1). \tag{2}$$

The last condition pays for rounding. The orientation construction then retains exactly P/r padded ports per symbol on each old side. Each child has at most

$$(2/r + o(1))M \tag{3}$$

wires. At a coordinate, exactly a fraction $1 - 1/r$ of ordered label pairs have unequal symbols and create crossing modes.

The crucial degree invariant is asymmetric. Each child places all retained parent tensors on one side and newly formed packet tensors on the other. The retained tensors acquire no ports, and each new packet tensor has degree at most $2L$. Thus at depth $d \geq 1$,

$$R_d \leq \left\lceil s^{9/10} \right\rceil, \qquad C_d \leq 2^{a_{d-1}+1}, \tag{4}$$

provided every preceding packet degree was below $s^{9/10}$. Our proofs check that premise explicitly. At the root both sides can have degree $\left\lceil s^{9/10} \right\rceil$.

3.2 Full crossing-block modes preserve the interface

For a retained interpolation node of valuation $i < u \leq a$, put $D_i = 2^{a-i}$. The joint-Cauchy packet constructs integral, group-local separated modes for the full crossing orientation [15]. Their certified valuations are

$$w_q = a + 1 - i + 4q - 2\,\mathrm{popcount}(q), \qquad 0 \leq q < D_i/2. \tag{5}$$

Each rank index has 2^i copies per orientation and there are two orientations per unequal-symbol packet. Consequently the number of copies per rank index is bounded by

$$c_i \leq \big(2(1 - 1/r) + o(1)\big)M/D_i. \tag{6}$$

The all-level decomposition is essential: the older annulus construction's slopes tend to three. A local slope-four calculation at $i = 0$ alone would not justify replacing three by four uniformly.

For clarity, the full-block construction uses slots $\sigma(j) = (j \bmod 2^i) + 2^{i+1}\lfloor j/2^i \rfloor$ and a finite telescoping Cauchy identity. At pivot $b = q2^i + t$, both separated factors have divisibility 2^{E_b}, where

$E_b = q + \nu_2(q!) = 2q - \mathrm{popcount}(q)$. The crossing scalar contributes $a + 1 - i$ more powers of two, giving (5). A mode reads columns in one group and supplies coefficients only to the other group. It changes neither child wires nor the degree invariant (4). This is why it composes with asymmetric packing without a hidden multiplicity penalty.

The retained tuple constraint is $\sum_q w_q \beta_q < p_i$. The exact source identity expresses the deformed contraction as a sum over such tuples of a product of r child contractions. Features and coefficients can be computed in polynomial bit time per elementary operation in the same coefficient rings as the source. Known factors of two are cancelled before unit inversion.

3.3 Sharp Hermite precision with a thin gap

The exact linear-taper count and recovery losses are prior results [5, 6]. Write

$$h = \left\lceil \frac{M(2L-1)+1}{L(2u-2+2^{1-u})} \right\rceil, \qquad q_i = h(u-i), \qquad p_i = p + 6hD_i + hu(a+1). \tag{7}$$

The total number of Hermite conditions is $hL(2u-2+2^{1-u})$, greater than the degree $M(2L-1)$ of the deformation. The displayed p_i is a sufficient working precision; exact smaller local losses could also be used.

Both new schedules set

$$g = \lceil \log_2 \lambda \rceil^2, \qquad a_d = \sum_{j=0}^{d} u_j + (d+1)g, \tag{8}$$

and stop at $H = \lceil s/\lambda \rceil$. At $O(\log \lambda)$ active depths, $a_d, u_d = O(\lambda)$ and the accumulated previous precision is bounded by $O(s\lambda^2 2^{a_{d-1}})$. Since $i < u_d$, $D_i \geq 2^{a_{d-1}+g+1}$. It follows that

$$\frac{p}{MD_i/u_d} = O(\lambda^4 2^{-g}) = o(1), \qquad \frac{hu_d(a_d+1)}{MD_i/u_d} = o(1).$$

When $u_d \to \infty$ uniformly, $h = (1+o(1))M/u_d$, and therefore

$$p_i \leq (6 + o(1))MD_i/u_d. \tag{9}$$

We prove the uniform growth and depth bound for each schedule below.

3.4 The resulting local cost

Lowering (5) to $\widehat{w}_q = 1 + 4q - 2\,\mathrm{popcount}(q)$ and extending the finite rank interval only enlarge the tuple set. The joint packet's weighted-partition estimate is

$$\sum_{q \geq 0} -\ln(1 - e^{-\theta \widehat{w}_q}) = \frac{\pi^2}{24\theta} + O\big(\log^2(2/\theta)\big).$$

Hence N_i, the number of retained tuples, satisfies $\ln N_i \leq \theta p_i + c_i \pi^2/(24\theta) + O(c_i \log^2(2/\theta))$. Optimize using the prescribed upper envelopes in (6) and (9). The optimizing $\theta \asymp \sqrt{u_d}/D_i$ tends uniformly to zero, and its error is lower order because $D_i \geq 2^g$. This yields

$$\log_2 N_i \leq \big(k(r) + o(1)\big)\frac{M}{\sqrt{u_d}}, \qquad k(r) = \frac{\pi\sqrt{2(1-1/r)}}{\ln 2}. \tag{10}$$

The factor four in the rank weights is already proved by prior work; the new contribution is the feasible allocation of the u_d and alphabets under the changing degree capacities.

Remark 3.1 (Creation-time representatives are retained). An existing tensor is evaluated with the precision at which its description was created. Later calls reuse its canonical integer representatives. Canonical lifting is not a ring homomorphism. The new schedules and kernel implementations do not replace that convention or recursively compose entry evaluators. Shared immutable descriptions preserve the source's oracle and space accounting.

4 A rational multistage schedule with polynomial space

A fixed alphabet is unnecessarily restrictive. The root is constrained by large degrees on both sides. Immediately below it, one side consists of fresh packet tensors, so a larger cumulative taper is possible. Later, a smaller alphabet allows a larger taper budget but gives slower size reduction. The schedule below makes that tradeoff explicit. Variable alphabets are already present in the round-two packet [17]; the result here is an auditable extension for the combined asymmetric, slope-four interface.

4.1 Parameters and exact margins

Use H, g, a_d from (8). Define $A_{-1} = 0$ and, for $0 \le d \le 5$, $u_d = \lfloor (A_d - A_{d-1})\lambda \rfloor$ according to the table.

Depth d	Alphabet r_d	Cumulative leading budget A_d	Individual t_d
0	13	27/2000	27/2000
1	14	21/800	51/4000
2	8	333/10000	141/20000
3	6	193/5000	53/10000
4	5	43/1000	11/2500
5	4	499/10000	69/10000

For $d = 6 + j$, put

$$r_d = 3, \qquad t_d = \frac{131}{40000}\left(\frac{3}{4}\right)^j, \qquad u_d = \lfloor t_d \lambda \rfloor . \tag{11}$$

The tail budget sums to 131/10000, so $\sum_{d \ge 0} t_d = 63/1000$. All scheduling operations are exact rational or integer arithmetic. The provided generator returns r_d, u_d, a_d, g without constructing 2^λ.

Lemma 4.1 (Uniform feasibility). *For all sufficiently large s, this schedule satisfies (2) and the precision assumptions at every active call. Active paths have $O(\log \lambda)$ depth; all retained tapers tend to infinity uniformly.*

Proof. Consider hypothetical active depths through $d_{\max} = \left\lceil \log_{10/7} \lambda \right\rceil + 7$. The cumulative gap is $O((\log \lambda)^3) = o(\lambda)$. Therefore $a_d \le (A_d + o(1))\lambda$ in the six fixed stages, and $a_d \le (63/1000 + o(1))\lambda$ in the tail.

At the root the dominant term is $Q_0^2 s^{9/10}$. Its power of s is $9/10 + (27/1000)\log_2 13 < 1$, certified by the integer inequality $13^{27} < 2^{100}$. The row term is smaller. Below the root, the retained-side power is $9/10 + A_d \log_2 r_d$. All required strict inequalities have exact certificates:

$$14^{21} < 2^{80}, \qquad 3 \cdot \frac{333}{10000} < \frac{1}{10}, \qquad 6^{193} < 2^{500},$$
$$5^{43} < 2^{100}, \qquad 2 \cdot \frac{499}{10000} < \frac{1}{10}, \qquad 3^{63} < 2^{100}.$$

Each finite stage and the whole tail have a fixed positive power margin. For the fresh side and rounding, the looser common estimates suffice:

$$2A_d \log_2 r_d < \frac{1}{5}, \qquad A_d, A_{d-1} \le \frac{63}{1000}.$$

Thus both $Q_d^2 C_d$ and $L_d Q_d^2$ are at most $s^{263/1000+o(1)}$ below the root. Every new packet degree is at most $s^{63/1000+o(1)} < s^{9/10}$, closing the degree invariant. Because every active M exceeds s/λ, these fixed power margins absorb all logarithmic factors and imply (2).

More quantitatively, (1) gives $P/M = 1 + O(1/\lambda) + s^{-\varepsilon+o(1)}$ uniformly for some fixed $\varepsilon > 0$. Consequently the exact rational guard

$$P/M \le 1 + 1/\left\lceil \sqrt{\lambda} \right\rceil$$

passes eventually. For sufficiently large λ it implies a child ratio at most $7/10$, including the slowest alphabet-three tail. No active path can exceed $d_{\max}$, validating the hypothetical depth range without a circular assumption. Products of the sharper padded overhead over that range are $1 + o(1)$. Write

$$v_0 = 1, \qquad v_{d+1} = v_d(2/r_d). \tag{12}$$

Then $M_d \le (1 + o(1))sv_d$ uniformly.

In the tail the smallest taper on that range is bounded below by

$$\lambda^{1-\ln(4/3)/\ln(10/7)+o(1)}.$$

The exponent is positive because $4/3 < 10/7$. Floors therefore have uniform relative error $o(1)$. The fixed stages have positive constant t_d. All retained u_d diverge, so (9) and the local entropy estimate apply uniformly. □

An implementation may use direct contraction for small inputs, zero tapers, or failed exact guards. These fallbacks preserve correctness. The lemma shows that they disappear from active calls eventually. The smallest margin in the power comparisons is tiny; this theorem makes no assertion of a practical input threshold for the complete contraction algorithm.

4.2 Summing the recursion and certifying the coefficient

Proposition 4.2. *The polynomial-space part of Theorem 1.1 holds with*

$$C_{\mathrm{poly}} = \sum_{d=0}^{5} \frac{k(r_d)v_d}{\sqrt{t_d}} + \frac{k(3)}{2730\sqrt{131/40000}} \frac{1}{1 - 4/(3\sqrt{3})}. \tag{13}$$

Proof. Along a recursive path, logarithms of tuple-enumeration costs add. By (10) and Lemma 4.1, the leading coefficient is bounded by $\sum_d k(r_d)v_d/\sqrt{t_d}$. At the start of the tail $v_6 = 1/2730$. Consecutive tail costs have ratio $(2/3)/\sqrt{3/4} = 4/(3\sqrt{3}) < 1$, which sums to (13). Uniform $o(1)$ perturbations change the strictly convergent tail by $o(1)$.

Per-call interpolation, arithmetic, group selection, and bookkeeping use $\mathrm{poly}(s)$ work on immutable handles. Input handling and original entry queries retain the fixed polynomial dependence on $s + \mathcal{L}$ from the source oracle interface. That entry-evaluation factor multiplies the total query workload, rather than being reintroduced at every recursion level. The $O(\log \lambda)$ levels therefore add only $O(\lambda \log \lambda)$ to the logarithm of the bookkeeping cost, which is negligible on the stated exponential

scale. Leaves cost $\exp(O(s/\lambda))$, initial tensor entries cost $\exp(O(s^{9/10}))$, and newly created entries cost $\exp(O(L \log s))$ with $L \le s^{63/1000+o(1)}$. All are $\exp(o(s/\sqrt{\lambda}))$. Depth-first enumeration retains polynomial space because each precision, label set, kernel description, and accumulator has polynomial size and the depth is logarithmic in λ. □

The checker bounds every radical by integer square-root intervals and uses rational intervals for π and $\ln 2$. It proves

$$63.781173756244 < C_{\mathrm{poly}} < 63.781173756245 < 64.$$

It also verifies the exact integer power certificates above. An independent coarser certificate, using $\pi < 22/7$ and $\ln 2 > 693/1000$, is

$$C_{\mathrm{poly}} < \frac{2297667966854588349649}{36002094515068500000} < 64.$$

Thus the convenient coefficient 64 is not a conclusion drawn from a floating-point search.

4.3 Comparison with the stronger prior family

For a fixed alphabet r with $c = 2/r$ and $A_0 = 1/(20 \log_2 r)$, the reviewed asymmetric analysis already optimizes root/tail allocation. After substituting slope-four modes, its boundary expression is

$$\frac{\pi\sqrt{2(1-1/r)}}{\ln 2\sqrt{A_0}} \left(1 + \frac{c}{(1-c^{2/3})^{3/2}}\right), \tag{14}$$

where the active root-cap and other feasibility premises apply. At $r = 16$ this is $66.192723693577\ldots$. The explicit inward-shifted schedule in the asymmetric packet gives about 66.6051 after the same kernel substitution. An integer scan of the admissible fixed-alphabet comparisons finds its minimum at 16 over the recorded range; we do not promote that scan to an optimality theorem over arbitrary schedules. Formula (13) is an explicit feasible improvement over these stronger combined certificates.

5 A faster schedule with subexponential memory

The full-block Cauchy packet already constructs a cached mode-graph algorithm: evaluate group functions, assemble three block tables, and contract their triangle by rectangular Strassen multiplication [15]. We retain that algorithm and give a new three-stage rational schedule. Combining it with asymmetric near-unit packing [16] proves the second part of Theorem 1.1. Its memory use is subexponential; it does not inherit the preceding section's polynomial-space bound.

5.1 The fully charged local recurrence

At a call with M edges, taper u, and r groups, let

$$K(M,u) = \frac{\pi\sqrt{2}}{\ln 2}\frac{M}{\sqrt{u}}.$$

Near-unit padding bounds the number of colors of each rank index on a specified directed pair of distinct groups by $(2+o(1))M/(r^2D)$. Combining the slope-four partition estimate with working precision $(6+o(1))MD/u$ gives the inherited selected-pair entropy bound

$$\sum_{e\in I} \log_2 |\mathcal{I}_e| \le \left(\frac{\sqrt{|I|}}{r} + o(1)\right) K(M,u).$$

Here $\mathcal{I}_e$ is the list of mode tuples of a prescribed exact weight on pair e. The weighted version replaces $|I|$ by the sum of squared pair weights. Exact pair-weight arrays are enumerated first, subject to the common precision budget. Their number is polynomial for fixed r. The original dynamic program provides exact counts and polynomial-delay indexing of each list; no completion-count oracle is assumed [15].

A group function depends on its $2(r-1)$ incident directed pairs. Caching it consequently requires at most $2^{(\alpha_r+o(1))K(M,u)}$ recursive child evaluations, where

$$\alpha_r = \frac{\sqrt{2(r-1)}}{r}.$$

For a block containing k groups, the counts of incident pairs and boundary pairs are $2rk-k^2-k$ and $2k(r-k)$, respectively. Their square roots divided by r determine the table-construction and storage coefficients. If the three unordered block-pair collections have cardinalities v_1, v_2, v_3, rectangular Strassen multiplication has coefficient

$$\beta_{\mathrm{matrix}} = \frac{\sqrt{v_1+v_2+v_3-[1-(\omega-2)^2]\min(v_1,v_2,v_3)}}{r}, \qquad \omega = \log_2 7.$$

This follows from its rectangular exponent $H_1+H_2+H_3-(3-\omega)\min H_j$ and the weighted entropy bound. The algorithm uses only ring operations, so it remains valid over the coefficient ring with its nilpotents and even modulus.

Write β_r for the larger of the construction and matrix coefficients, and j_r for the boundary-storage coefficient. The three alphabets used below give

$$B_* := \frac{\sqrt{2\bigl(2+(\omega-2)^2\bigr)}}{3},$$

r	Block sizes	α_r	β_r	j_r
7	$(2,2,3)$	$\sqrt{12}/7$	$\sqrt{30}/7$	$\sqrt{24}/7$
9	$(3,3,3)$	$4/9$	B_*	$2/3$
6	$(2,2,2)$	$\sqrt{10}/6$	B_*	$2/3$

For $r=7$, table construction dominates the matrix product: the latter's squared numerator is $24+8(\omega-2)^2<30$. The exact integer inequality $7^{16}<2^{45}$ proves $\omega<45/16$, which also gives $B_*<77/100$.

Suppressing polynomial factors, the correct recurrence is

$$T_d \le 2^{(\alpha_{r_d}+o(1))K_d}\, r_d T_{d+1} + 2^{(\beta_{r_d}+o(1))K_d}, \qquad K_d = K(M_d,u_d). \tag{15}$$

Thus all recursive group evaluations are charged. Block-table enumeration uses cached values, rather than multiplying its entire work by another child-contraction cost. The inherited sequential-record ordering, rewind charging, and matrix reblocking arguments give this bound in bit operations, without assuming unit-cost access to an exponential table [15].

5.2 An exact three-stage schedule

Set $\lambda = \lceil \log_2 s \rceil$, $H = \lceil s/\lambda \rceil$, $g = \lceil \log_2 \lambda \rceil^2$, and $a_d = \sum_{j \le d} u_j + (d+1)g$. Choose

$$\begin{aligned} r_0 &= 7, & u_0 &= \left\lfloor \frac{89\lambda}{5000} \right\rfloor, \\ r_1 &= 9, & u_1 &= \left\lfloor \frac{137\lambda}{10000} \right\rfloor, \\ r_{2+j} &= 6, & u_{2+j} &= \left\lfloor \frac{923\lambda}{250000} \left(\frac{12}{25}\right)^j \right\rfloor \quad (j \ge 0). \end{aligned} \tag{16}$$

The leading cumulative taper after the second stage is $63/2000$. The tail budget is $71/10000$, making the full leading cumulative budget $193/5000$. The parameter generator computes these rational floors and cumulative sums exactly, without constructing an instance of size 2^λ.

We verify feasibility before assuming a recursion-depth bound. Consider hypothetical active depths through $\left\lceil \log_{5/2} \lambda \right\rceil + 3$. Their cumulative gaps are $O((\log \lambda)^3) = o(\lambda)$. The three exact witnesses

$$7^{89} < 2^{250}, \qquad 9^{63} < 2^{200}, \qquad 6^{193} < 2^{500}$$

prove the root constraint $9/10 + 2(89/5000)\log_2 7 < 1$ and the retained-side constraints $9/10 + (63/2000)\log_2 9 < 1$ and $9/10 + (193/5000)\log_2 6 < 1$. Below the root, fresh column degrees are at most $2^{a_{d-1}+1}$. The fresh-side and padding exponents are therefore less than $1/5 + 193/5000 < 1$; all new degrees remain below $s^{9/10}$. These fixed power margins absorb both the logarithmic factors and $M^{-1} < \lambda/s$ in the three packing conditions.

Uniformly, the padded-mass ratio is $1 + O(1/\lambda) + s^{-\epsilon+o(1)}$ for some fixed $\epsilon > 0$. Hence it eventually passes the exact actual-mass guard, and every child has at most $2M/5$ edges. This proves that no active path exceeds the hypothetical depth range and closes the induction. Tail tapers grow uniformly as a positive power of λ, because $25/12 < 5/2$. Their floors are negligible. The thin-gap precision proof and the uniform slope-four entropy estimate consequently apply at every active node. Small inputs and failed finite guards retain the direct-contraction fallback.

5.3 Time and memory, including the entire tail

Let $k = \pi\sqrt{2}/\ln 2$. After suppressing the common scale $s/\sqrt{\lambda}$, the first three upper envelopes for K_d are

$$\begin{aligned} \widehat{K_0} &= \frac{k}{\sqrt{89/5000}} = 48.0429214176\ldots, \\ \widehat{K_1} &= \frac{(2/7)k}{\sqrt{137/10000}} = 15.6462794092\ldots, \\ \widehat{K_2} &= \frac{(4/63)k}{\sqrt{923/250000}} = 6.6977356903\ldots. \end{aligned}$$

The subsequent envelopes form a geometric sequence with ratio

$$\gamma = \frac{1/3}{\sqrt{12/25}} = \frac{5}{6\sqrt{3}} < 1.$$

Products of the padding errors across $O(\log \lambda)$ levels are $1 + o(1)$, so these are uniform leading envelopes for the actual recursion.

Unrolling (15), the tail contributes its previously paid prefix plus

$$\widehat{K_2}\max\left\{B_*, \frac{\alpha_6}{1-\gamma}\right\}.$$

Indeed a terminal matrix operation at tail depth j has coefficient $\widehat{K_2}[\alpha_6(1-\gamma^j)/(1-\gamma) + B_*\gamma^j]$; its supremum is the larger endpoint. Here the infinite cache endpoint $\alpha_6/(1-\gamma) > 1$ dominates B_*. The complete time coefficient is therefore bounded by the maximum of

$$\begin{aligned}\beta_7\widehat{K_0} &= 37.5917025555\ldots,\\ \alpha_7\widehat{K_0} + B_*\widehat{K_1} &= 35.7860204992\ldots,\\ \alpha_7\widehat{K_0} + \alpha_9\widehat{K_1} + \frac{\alpha_6\widehat{K_2}}{1-\gamma} &= 37.5321972011\ldots.\end{aligned}$$

The root is strictly largest. Rational square-root intervals certify both comparisons after cancelling the common positive factor k. In particular, this comparison pays the entire infinite cached tail, rather than testing only a few recursion levels. Simplifying the root expression gives

$$\boxed{C_{\text{cache}} = \frac{\pi}{\ln 2}\sqrt{\frac{300000}{4361}}}.$$

The storage envelopes are $j_7\widehat{K_0}$, $j_9\widehat{K_1}$, and $j_6\widehat{K_2}\gamma^j$. Their maximum is the root value. An active stack stores a sum of polynomially many tables, whose leading exponential coefficient is their maximum; it is not a product of those storage sizes. Thus

$$\boxed{S_{\text{cache}} = \frac{\pi}{\ln 2}\sqrt{\frac{240000}{4361}}} = \sqrt{4/5}\,C_{\text{cache}}.$$

Polynomial-bit records, sequential sorting and reblocking, and the polynomial number of exact weight arrays preserve both coefficients. Interpolation and other per-level polynomial work add only $O(\lambda \log \lambda)$ to the time logarithm. Leaves, original entries, and new packet entries have the same lower-order bounds as in the preceding proof. Immutable descriptions retain their creation-time representatives; no recursively composed entry oracle or altered lifting rule is introduced.

The exact checker certifies

$$\begin{aligned}37.591702555541 < C_{\text{cache}} < 37.591702555542,\\ 33.623040921657 < S_{\text{cache}} < 33.623040921658.\end{aligned}$$

These are rationally certified enclosures, using Machin's formula and a positive series for $\ln 2$. Consequently eventual coefficients 37.6 for time and 34 for space are valid. No practical crossover size follows from these asymptotic margins.

6 Two exact improvements to executable packing kernels

The schedule theorems concern asymptotic contraction certificates. The next results operate on explicit finite graphs and do not need those asymptotic packing margins to hold. They preserve the corresponding original routines' choices exactly, so any valid prior packing certificate continues to apply. Incremental conditional expectations and common-denominator arithmetic are classical techniques; our novelty claim is the specified implementation and proved improvement relative to the reviewed executable corpus.

6.1 Incremental products for matching potentials

Partition the bipartite edge occurrences into matchings $E_1, \dots, E_K$. Parallel edges remain distinct and occur in different matchings. Each vertex has one of Q labels. For a partially assigned vertex v, let $p_v(a)$ be its assigned-label indicator, or $1/Q$ if it remains unassigned. In this subsection $\eta \geq 1$ denotes the reference's concentration parameter, called `lam` in code; it need not equal a tensor-size logarithm. Put $t = 1 + 1/\eta$ and let h_j be the reference thresholds. Define

$$P_{j,a,b} = t^{-h_j} \prod_{(u,v)\in E_j} \left(1 + \frac{p_u(a)p_v(b)}{\eta}\right), \qquad \Phi = \sum_{j,a,b} P_{j,a,b}. \tag{17}$$

Because edges within a matching have disjoint endpoints, this is exactly the conditional expectation of $\sum_{j,a,b} t^{X_{j,a,b}-h_j}$. Independence between different matchings is unnecessary: expectations add. The latest matching packet evaluates (17) afresh for every candidate label at every vertex [18].

Theorem 6.1 (Same choices with incremental matching products)**.** *For an explicit graph with M edge occurrences and V incident vertices, the implementation* ***`label_graph_incremental`*** *returns the same first-fit matching partition, labels, thresholds, loads, and complete selected potential trace as the pinned reference. Its update phase uses $O(MQ^2 + VQ)$ rational operations, compared with $\Theta(VMQ^3)$ elementary product operations in the nonempty reference's repeated evaluations. Initialization, coloring, bit costs, and extra storage are charged below.*

Proof. Suppose an unassigned left vertex u is processed. Each incident matching contains exactly one edge (u, v). For every opposite label b with $p_v(b) > 0$, set

$$f_b = 1 + \frac{p_v(b)}{Q\eta}, \qquad R_a = P_{j,a,b}/f_b.$$

The divisor is positive. In this slice the old contribution is $\sum_a P_{j,a,b}$. If u receives candidate label c, the new contribution is

$$\sum_a R_a + \frac{p_v(b)}{\eta} R_c.$$

Thus all candidates share a base, and only the last correction depends on c. Summing affected slices gives

$$\begin{aligned} B &= \Phi + \sum_{\text{affected slices}} \left(\sum_a R_a - \sum_a P_{j,a,b}\right), \\ S_c &= \sum_{\text{affected slices}} \frac{p_v(b)}{\eta} R_c, \qquad \Phi_c = B + S_c. \end{aligned} \tag{18}$$

No slice is changed twice within a matching because E_j is a matching. The right-vertex rule interchanges the indices. Dividing the affected table entries by their old factors, then multiplying the chosen entries by $1 + p_v(b)/\eta$, restores (17) exactly. The averaging identity $\sum_c \Phi_c = Q\Phi$ follows either from the formulas or from conditional expectation and is checked exactly by the implementation.

Candidate values and smallest-label tie breaking therefore equal those from recomputation. Induction gives every selected trace value and final load. The endpoint-occupancy sets used in first-fit coloring make the same first available matching choice as the reference's edge scans; input order is retained. This proves full-output equality, including tuple-pair and list-pair edge representations.

When an edge's first endpoint is processed, its other endpoint is uniform; there are Q affected slices of Q cells. When the second endpoint is processed, there is one affected slice of Q cells. Summing

over edge occurrences gives $O(M(Q^2+Q))$ rational operations, plus $O(VQ)$ candidate summation and selection operations. In contrast, the reference has VQ full potential evaluations, each scanning M edge factors in Q^2 cells. □

Costs outside the update phase. The initial value of every cell for matching j is

$$t^{-h_j}\left(1+\frac{1}{\eta Q^2}\right)^{|E_j|}.$$

Binary powering costs $O(K(\log(M+1)+\log(h_{\max}+1)))$ rational multiplications; the tables require $O(KQ^2)$ initial assignments. First-fit coloring uses $O(MK)$ endpoint-set membership tests, and vertex sorting uses $O(V\log V)$ comparisons. The final product-invariant check costs $O(KQ^2)$. The maintained tables contain KQ^2 rational values, in addition to graph information, Q scores, and V selected trace values. This deliberately spends more product storage than recomputation, while remaining polynomial in the application parameters.

A common denominator for the conditional products divides $(\eta+1)^{h_{\max}}\eta^{m_{\max}}Q^{2m_{\max}}$, where $m_{\max}=\max_j|E_j|$. Product bit lengths are therefore $O((h_{\max}+m_{\max})\log(\eta Q+2))$, with logarithmic overhead for sums. Multiplication, division, and greatest-common-divisor costs must be paid at those lengths. The theorem's operation count does not assume constant-time large-integer arithmetic. Bit sizes are polynomial for explicit graphs and the concentration parameters used by the reviewed contraction schedules.

6.2 Common-denominator integer scores for column balancing

The two-stage exponential-potential construction first groups rows, producing nonnegative integer weights $w_{r,v}$ from row group r to column vertex v. There are Q row groups, Q possible column labels, and N column vertices. Let $z=a/b>1$ be in lowest terms. The earlier packet refinement already caches suffix products and row-power sums, achieving $O(NQ^2)$ dense arithmetic work [13]. We do not claim that order or the idea of caching as new. Our change removes repeated rational normalization from candidate scoring.

At time t, let $L_{r,c}$ be the assigned weight in cell (r,c), $S_r=\sum_c L_{r,c}$, $W_r=\sum_v w_{r,v}$, and $W=\max_r W_r$. Set

$$\begin{aligned} C_w &= (Q-1)b^w+a^w,\\ A_r &= b^{W-W_r}\prod_{v\ \text{unassigned}} C_{w_{r,v}}, \qquad H_{r,c}=a^{L_{r,c}}b^{S_r-L_{r,c}}. \end{aligned} \tag{19}$$

All these quantities are nonnegative integers. The unnormalized conditional potential has the exact representation

$$\Phi=\frac{\sum_{r,c}A_rH_{r,c}}{Q^{N-t}b^W}. \tag{20}$$

Indeed the ordinary row-cell expectation is $z^{L_{r,c}}\prod_{v\ \text{unassigned}}(Q-1+z^{w_{r,v}})/Q$; clearing the common denominator gives (20). An optional common threshold multiplier z^{-B_0} has no effect on choices.

For the next column, put $w_r=w_{r,v}$ and divide $A'_r=A_r/C_{w_r}$ exactly. Candidate c then has numerator

$$D_c=\sum_r A'_r\left[b^{w_r}\sum_h H_{r,h}+(a^{w_r}-b^{w_r})H_{r,c}\right] \tag{21}$$

over the common denominator $Q^{N-t-1}b^W$. Moreover

$$\sum_c D_c = \sum_{r,c} A_r H_{r,c}.$$

Choose the least integer numerator, breaking ties by label index. Multiply the chosen $H_{r,c}$ by a^{w_r} and each other $H_{r,h}$ by b^{w_r}. These updates restore (19). By induction, every chosen label and every reconstructed rational candidate potential equals the reference. Zero weights, empty columns, and $Q = 1$ obey the same identities.

Dense arithmetic remains $O(NQ^2)$. The core score computations use integers and divisions known to be exact; rational reduction is deferred to the requested diagnostic output. The supplied implementation still constructs the full candidate trace, so its measured gains do not arise from omitting that work. It removes the suffix-product table and uses $O(Q^2)$ main update state, excluding input, power dictionaries, and the explicitly requested $O(NQ)$ candidate output objects.

Since $C_w \le Qa^w$, the main numerator bit length is at most $O(N \log Q + W \log a + \log Q)$. Threshold normalization adds $O(B_0 \log a)$ bits. Exact integer divisions and output rational reductions are charged normally. These bounds concern explicit edge multiplicities; we make no polynomial-in-log-weight claim for compressed enormous weights. Unlike the matching theorem, this result does not assert a better dense asymptotic operation count than the strongest cached reference. It supplies an exact representation change and a measured implementation gain.

7 Reproducible tests and measured performance

All Python components use the standard library. The reference implementations are copied verbatim from the reviewed packets, with archive/member paths and SHA-256 identities in `code/reference/provenance.json`. The mathematical checker suite verifies those identities before importing them. The full reproduction uses no network and does not rebuild a proof assistant.

7.1 Exact validation

Both ordinary Python and `python -O` pass the following checks:

- **Column kernels:** 932 instances, including exhaustive small Boolean weight matrices, empty columns, zero weights, ties, seeded integer weights, two rational bases, and different thresholds. Each of two incremental implementations matches 9,828 exact candidate potentials and every output field of the direct reference.
- **Matching kernel:** 399 instances, including every simple 2×3 graph, seeded multigraphs, parallel edges, $Q = 1, 2, 3$, and both accepted edge-pair representations. All assignments, matching partitions, thresholds, loads, and 1,920 selected trace values match exactly.
- **Independent expectation check:** 128 prefix expectations are computed by averaging 1,366 full assignments using final cell loads and the defining exponential potential. This check uses neither implementation's conditional-product formula and catches a possible shared formula error.
- **Schedules:** integer power witnesses, radical enclosures, convergence margins, infinite-tail dominance, and 216 exact parameter-generator cases pass. The two coefficient intervals are proved with rational arithmetic, not estimated from large-parameter timing samples.

The package replay runs six checker processes and compares the mathematical outputs across ordinary and optimized Python and with the shipped results. Only execution flags, elapsed times, and

interpreter metadata are omitted from those comparisons. Explicit failure checks remain active under `-O`. Finite tests corroborate the local implementation identities; the all-size conclusions are proved in Sections 4–5.

7.2 Benchmark protocol

Measurements use one CPython 3.12.14 process on Linux x86-64 and `perf_counter_ns`. Each fixture has a recorded seed and five repetitions, alternating baseline and improved execution order. The result of every timed execution is checked for exact equality after timing. Tables show medians of the full call, including graph preprocessing. Raw samples, interpreter/platform metadata, fixture parameters, and implementation hashes are retained in `results/benchmarks.json`.

These are seeded synthetic graph-packing fixtures, not SAT formulas. They were chosen to compare exact kernel semantics at manageable sizes. Changing Q, graph structure, or the arithmetic backend can change the ratio. No statistical claim about a population of SAT instances follows from this set.

Matching-potential kernel: seconds per complete labeling call				
Edges M	Labels Q	Reference	Incremental	Ratio
16	2	0.005282	0.000922	5.73×
32	4	0.147977	0.005195	28.48×
64	4	0.543776	0.011181	48.63×
128	4	2.131277	0.021639	98.49×
96	8	7.480087	0.056324	132.81×

The reference here is the newest matching archive's `refinement.py:label_graph`. Its analytic certificate is unchanged. The incremental routine returns the same complete dictionary, including its selected potential trace. The extra product table is included in the new routine's implementation; the comparison does not claim equal peak space.

Column kernel: seconds including the same graph preprocessing					
Columns N	Edges M	Labels Q	Cached suffix	Integer scores	Ratio
16	64	4	0.001113	0.000284	3.92×
32	128	4	0.002900	0.000593	4.89×
64	256	8	0.016422	0.002153	7.63×
128	512	8	0.042958	0.006298	6.82×
256	1024	16	0.357637	0.032936	10.86×

This stronger comparator already uses cached suffix products and row-power sums. The integer implementation additionally constructs all candidate fractions internally, then returns the reference's selected-trace interface. Comparing only with the slower direct potential would overstate the new work.

The observed ranges are therefore 5.73–132.81 for matching and 3.92–10.86 for columns on these fixtures. They demonstrate concrete local improvement alongside the exact identities. They do not establish those factors for other inputs, for a compiled implementation, or for an end-to-end subexponential SAT solver. No such complete solver is supplied by this work or by the benchmark references.

8 Integration, remaining limits, and further research

8.1 How the results fit into the existing framework

For the analytic improvement, retain the supplied deformation and recovery identities and substitute the schedules from `code/schedules.py`. Use the asymmetric whole-vertex packing bound, keep the two side-degree bounds separately, and use full crossing-block modes at every retained valuation level. Maintain each tensor's creation precision. The polynomial and cached schedules are separate resource choices; the cached mode-graph algorithm must explicitly charge group evaluations and store block tables.

For the matching implementation, replace the reference `label_graph` call with `label_graph_incremental`. Its complete result agrees on valid explicit graph inputs. For two-stage column balancing, use `derandomize_integer` on the row-group weight matrix; the included benchmark wrapper demonstrates the unchanged `Labeling` interface. The functions preserve mathematical choices, so no new tail inequality is needed to reuse the corresponding packing certificate. A production implementation may stream diagnostics rather than retain every candidate, but the benchmarks here include them.

These are integration-ready components and precise source-level replacement instructions. The project does not yet contain a complete contractor to which we could honestly report a tested end-to-end patch. The archive therefore supplies component implementations and proofs without claiming an implemented full counting pipeline. No remote repository write was required for this review and delivery.

8.2 Preserve the best applicable previous branch

The new general high-degree coefficients should not displace stronger specialized bounds. For example, the prior low-degree coefficient 31.0641 combines with slope-four modes to give about 26.9023 under its stronger degree premise. Similarly the density-sensitive reduction and the scalar parity lift remain available. A dispatcher can select a certificate whose premises have been verified; a finite portfolio can alternatively interleave correct deterministic algorithms with constant-factor scheduling overhead. Its upper bound is the minimum of the applicable running-time bounds, up to that overhead. For a polynomial-space portfolio, retain only polynomial-space branches.

Thus the new contributions improve particular general-interface certificates and compatible executable kernels while preserving the existing specialized envelope. They do not numerically order an independent manuscript with an unspecified constant. Nor does a smaller normalized wire coefficient supply a variable-level coefficient without tracking the SAT reduction's wire count.

8.3 Why the general logarithmic order has not changed

The inherited mode pool still has linearly growing rank weights, and the packing conditions still permit taper u only on a logarithmic scale. Combining $c \asymp M/D$ with $p \asymp MD/u$ gives tuple entropy on the $M/\sqrt{u}$ scale. Our finite alphabet schedules alter its leading coefficient, not those two structural facts. For fixed clause width, the strongest supported density envelope in Section 2 is therefore retained; in particular the unrestricted bound still has exponent order $n \log\log n/\sqrt{\log n}$.

The conditional $n/\log n$-type proposals need a faster uniform summation or factorization interface that is absent from the reviewed corpus. This article does not construct it. The full-block Smith spectrum is a local obstruction to changing its linear rank-weight growth by an invertible integral

basis change; it is not a lower bound for SAT or for other representations.

8.4 Specific next research questions

1. **Optimize the changing capacity constraints rigorously.** The rational schedules provide witnesses, not global optima. A useful next result is an interval-certified optimizer for the recurrence with state equal to the cumulative packet exponent, current degree bounds, and available memory. It should bound the infinite tail and the admissible alphabet range, rather than rely on a finite numerical scan.
2. **Use actual real-port restrictions of Cauchy blocks.** The exact Smith valuations concern the full padded orientation. A polynomial-time method that finds a smaller integral separation for the real-port submatrix, while preserving source/target locality and entry costs, could reduce costs beyond another fixed schedule optimization. Any resulting claim must include the cost of finding and applying the separation.
3. **Clear denominators for matching updates as well.** The matching routine currently uses exact fractions in its incremental product tables. A shared-denominator representation analogous to the column method could reduce normalization costs further. It must avoid enormous unreduced common integers and account for adaptive thresholds and sparse incident sets. This is a concrete next implementation experiment.
4. **Improve the time-memory tradeoff continuously.** The cached algorithm stores three block tables. Partial caching, unequal blocks, or a schedule that spends memory only at selected depths may offer better bounds for a fixed memory budget. A valid recurrence must charge recomputation and sequential access rather than assume free table lookup.
5. **Formalize the local invariants before the complete pipeline.** The matching-product update, common-denominator identity, and exact rational schedule guards are small independent targets. A full formal endpoint would also need inherited representation semantics, the bit-cost model, and the uniform depth/precision induction. Passing finite Python tests is useful evidence for those interfaces, not a replacement for such proofs.

Reproduction and provenance. Run `python3 run_checks.py` for the six-process exact replay, `python3 code/benchmark_kernels.py` for fresh timing measurements, and `bash build_report.sh` to rebuild this PDF with pdfLaTeX. The source inventory, reference hashes, benchmark seeds and raw samples are included. The benchmark records describe the delivered implementation hashes; new runs may change timing metadata without changing mathematical outputs. The report's proof dependency boundary remains explicit throughout.

References

[1] OpenAI, *The Exponential Time Hypothesis is False*, October 25, 2026. Pinned manuscript and quantitative appendices:

https://github.com/openai/math/tree/7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026.

Particularly `quantitative-contraction.tex`: packet orientation, Hermite recovery, the recursive identity, and child entry/space bounds.

[2] Vladimir Reshetnikov, ProveIt SubexponentialSAT project index, reviewed snapshot:

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/README.md.

The incoming archives below are at the same frozen commit.

[3] Claude (Anthropic), *A $2^{o(n)}$ time algorithm for 3-SAT*, October 26, 2026, 49-page manuscript:

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/claude-2o-n-3sat/eth.pdf.

[4] *Factorial Cauchy modes for the subexponential 3-SAT algorithm.* Research packet filed in ProveIt, October 26, 2026.

https://github.com/VladimirReshetnikov/ProveIt/tree/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/cauchy_mode_improvement.

[5] *Quantitative improvements to the exact-counting contraction algorithm.* Research packet filed in ProveIt, October 26, 2026.

https://github.com/VladimirReshetnikov/ProveIt/tree/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/eth_contraction_improvements.

[6] *Sharper precision for subexponential 3SAT.* Research packet filed in ProveIt, October 26, 2026.

https://github.com/VladimirReshetnikov/ProveIt/tree/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/3sat_precision_refinement.

[7] *Improving the clause-density dependence of exact k-SAT counting.* Research packet filed in ProveIt, October 26, 2026.

https://github.com/VladimirReshetnikov/ProveIt/tree/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/sat_density_improvement.

[8] *A density-sensitive refinement of exact k-SAT counting.* Research packet filed in ProveIt, October 26, 2026.

https://github.com/VladimirReshetnikov/ProveIt/tree/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/eth_density_refinement.

[9] *Refining the October 25, 2026 subexponential SAT algorithm.* Research packet filed in ProveIt, October 26, 2026.

https://github.com/VladimirReshetnikov/ProveIt/tree/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/3sat_refinement.

[10] *Asymptotic improvement analysis for the October 2026 3-SAT preprint.* Research packet filed in ProveIt, October 26, 2026.

https://github.com/VladimirReshetnikov/ProveIt/tree/d098ba436b33df19365a44dd178694ff52845d19/Computability/SubexponentialSAT/Research/3sat_asymptotic_analysis.

[11] *Balanced collision packing for exact counting*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/eth_balanced_packing_2026-10-26.zip.

[12] *Higher moments and economical precision gaps for subexponential SAT*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/eth_high_moment_2026-10-26.zip.

[13] *Sharper packet packing for subexponential exact counting*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/eth_packet_refinement_2026-10-26.zip.

[14] *ETH contraction refinement*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/eth_recursive_refinement_2026-10-26.zip.

[15] *Full crossing-block Cauchy modes*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/eth_joint_cauchy_2026-10-26.zip.

[16] *Asymmetric packet balancing for subexponential SAT*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/eth_asymmetric_packet_balancing_2026-10-26.zip.

[17] *SubexponentialSAT review and contraction refinement*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/3sat_round2_refinement_20261027.zip.

[18] *Matching packing and thin precision gaps*. Incoming research archive at the reviewed snapshot.

https://github.com/VladimirReshetnikov/ProveIt/blob/d098ba436b33df19365a44dd178694ff52845d19/docs/incoming/ETH_Matching_Packing_Thin_Gaps_2026-10-27.zip.